SIMATS ENGINEERING

CO3_ASSESSMENT TOOL3_ Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**192511426**
Name	**K..Devarshan**

COURSE OUTCOME COVERED

CO3: *Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Q1.

Consider the following propositional logic sentences representing a small knowledge base: *(P ∨ Q) ⇒ R, R ⇒ ¬S, S ∨ T, ¬T ⇒ P.* (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of ∨ over ∧). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α. Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of KB ∧ ¬α, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) *Knows(John, x)* and *Knows(John, Jane)*; (ii) *Knows(x, Elizabeth)* and *Knows(y, x)*; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

Q5.
Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree / goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

*I _**(K.B.Devarsham/192511426)**_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	**10**
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	**10**
Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier	Computes the MGU correctly for most pairs with only minor errors; failure case	Partial implementation of unification; produces an incorrect MGU for some pairs	Little or no correct implementation of the unification algorithm.	**10**

	for all test pairs, including correctly identifying the unification-failure case.	handled with a small issue.	or does not correctly detect the failure case.		
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	**10**
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	**10**

Q1. Propositional Logic to CNF

(a) Manual Conversion to CNF

1. $(P \vee Q) \Longrightarrow R$
   - **Implication elimination:** $\neg(P \vee Q) \vee R$
   - **Negation inward (De Morgan's):** $(\neg P \wedge \neg Q) \vee R$
   - **Distribution ($\vee$ over $\wedge$):** $(\neg P \vee R) \wedge (\neg Q \vee R)$
2. $R \Longrightarrow \neg S$
   - **Implication elimination:** $\neg R \vee \neg S$
3. $S \vee T$
   - Already in CNF: $S \vee T$
4. $\neg T \Longrightarrow P$
   - **Implication elimination:** $\neg(\neg T) \vee P$
   - **Negation inward (Double negation):** $T \vee P$

(b) Python Implementation

```
from sympy.logic.boolalg import to_cnf
from sympy.abc import P, Q, R, S, T
from sympy import Implies, Or, And, Not

def convert_to_cnf(sentences):
    print("--- CNF Conversion Results ---")
    for i, sentence in enumerate(sentences):
        cnf_form = to_cnf(sentence, simplify=True)
        print(f"Sentence {i+1}: {sentence}")
        print(f"CNF Form  : {cnf_form}\n")

# Define the knowledge base sentences based on the question
sentence1 = Implies(Or(P, Q), R)
sentence2 = Implies(R, Not(S))
sentence3 = Or(S, T)
sentence4 = Implies(Not(T), P)

kb = [sentence1, sentence2, sentence3, sentence4]

convert_to_cnf(kb)
```

Output:

Plaintext

```
--- CNF Conversion Results ---
Sentence 1: Implies(P | Q, R)
CNF Form  : (R | ~P) & (R | ~Q)

Sentence 2: Implies(R, ~S)
CNF Form  : ~R | ~S

Sentence 3: S | T
CNF Form  : S | T

Sentence 4: Implies(~T, P)
CNF Form  : P | T
```

Q2. PL-RESOLUTION

Knowledge Base (Wumpus World snippet):

Let $B_{1,1}$ = Breeze in (1,1), $P_{1,2}$ = Pit in (1,2), $P_{2,1}$ = Pit in (2,1).

- **Rule 1:** A breeze in (1,1) means a pit in (1,2) or (2,1). $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- **Fact 1:** There is no breeze in (1,1). $\neg B_{1,1}$
- **Query (α):** Is there no pit in (1,2)? $\neg P_{1,2}$
-

(a) CNF Form of $KB \wedge \neg\alpha$

- Rule 1 translates to 3 CNF clauses: $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1})$, $(\neg P_{1,2} \vee B_{1,1})$, $(\neg P_{2,1} \vee B_{1,1})$
- Fact 1 translates to: $(\neg B_{1,1})$
- Negated Query ($\neg\alpha$): $\neg(\neg P_{1,2}) \equiv (P_{1,2})$

(b) Implementation & Complete Resolution Steps

Python

```
def resolve(ci, cj):
    """Returns the set of all possible clauses obtained by resolving two clauses."""
    resolvents = set()
    for literal in ci:
        neg_literal = literal[1:] if literal.startswith('~') else '~' + literal
        if neg_literal in cj:
            # Create resolvent by combining clauses and removing the complementary literals
            res = (ci - {literal}) | (cj - {neg_literal})
            resolvents.add(frozenset(res))
    return resolvents

def pl_resolution(kb, alpha):
    """Returns True if KB entails alpha using resolution by refutation."""
    # Negate alpha (assume alpha is a single literal for simplicity in this example)
    neg_alpha = frozenset([alpha[1:] if alpha.startswith('~') else '~' + alpha])
    clauses = kb.copy()
    clauses.add(neg_alpha)

    new_clauses = set()
    step = 1

    while True:
        clauses_list = list(clauses)
        n = len(clauses_list)
        for i in range(n):
            for j in range(i + 1, n):
                resolvents = resolve(clauses_list[i], clauses_list[j])
                for res in resolvents:
                    if not res: # Empty clause
                        print(f"Step {step}: Resolved {set(clauses_list[i])} and {set(clauses_list[j])}")
                        print("Result: Empty Clause [] -> Contradiction found.")
                        return True
                    if res not in clauses and res not in new_clauses:
                        print(f"Step {step}: Resolved {set(clauses_list[i])} and {set(clauses_list[j])} ->
{set(res)}")
                        new_clauses.add(res)
                        step += 1

        if new_clauses.issubset(clauses):
            return False # No new clauses generated
        clauses.update(new_clauses)

# KB Clauses
kb = {
    frozenset(['~B11', 'P12', 'P21']),
    frozenset(['~P12', 'B11']),
    frozenset(['~P21', 'B11']),
    frozenset(['~B11'])
}
query = '~P12'
```

```
print("Starting Resolution...")
result = pl_resolution(kb, query)
print(f"Entailment Result: {result}")
```

(c) Trace & Final Entailment Result

1. **Step 1:** Resolve negated query $\{P_{1,2}\}$ with $\{\neg P_{1,2}, B_{1,1}\} \Longrightarrow \{B_{1,1}\}$.
2. **Step 2:** Resolve the newly derived $\{B_{1,1}\}$ with the KB fact $\{\neg B_{1,1}\} \Longrightarrow$ Empty Clause [].
3. **Result:** Contradiction reached. Therefore, the KB entails $\neg P_{1,2}$.

Q3. MGU Algorithm

Implementation of UNIFY:

Python

```
def is_variable(x):
    return isinstance(x, str) and x.islower()

def unify(x, y, theta):
    if theta is None: return None
    elif x == y: return theta
    elif is_variable(x): return unify_var(x, y, theta)
    elif is_variable(y): return unify_var(y, x, theta)
    elif isinstance(x, list) and isinstance(y, list):
        if len(x) != len(y): return None
        return unify(x[1:], y[1:], unify(x[0], y[0], theta))
    else:
        return None

def unify_var(var, x, theta):
    if var in theta: return unify(theta[var], x, theta)
    elif x in theta: return unify(var, theta[x], theta)
    elif occurs_check(var, x, theta): return None  # Occurs check
    else:
        # Add new substitution to theta
        theta[var] = x
        return theta

def occurs_check(var, x, theta):
    if var == x: return True
    elif isinstance(x, list):
        return any(occurs_check(var, arg, theta) for arg in x)
    elif x in theta:
        return occurs_check(var, theta[x], theta)
    return False
```

Testing the Pairs:

- **(i) Knows(John, x) and Knows(John, Jane)**
  - **Input:** ['Knows', 'John', 'x'], ['Knows', 'John', 'Jane']
  - **Output MGU:** {'x': 'Jane'}
- **(ii) Knows(x, Elizabeth) and Knows(y, x)**
  - **Input:** ['Knows', 'x', 'Elizabeth'], ['Knows', 'y', 'x']
  - **Output MGU:** {'x': 'Elizabeth', 'y': 'Elizabeth'}
- **(iii) Failing Pair: Loves(x, Father(x)) and Loves(y, y)**

- **Input:** ['Loves', 'x', ['Father', 'x']], ['Loves', 'y', 'y']
- **Output:** Failure (None)
- **Explanation:** The algorithm first binds x to y. Next, it attempts to unify ['Father', 'x'] with y. Because x is already bound to y, this is equivalent to unifying ['Father', 'y'] with y. The **occurs check** detects that the variable y is present *inside* the term Father(y). Allowing this unification would create an infinite, recursive binding loop ($y = Father\left(Father\left(Father(\dots)\right)\right)$). Therefore, it correctly returns failure.

Q4. First-Order Resolution
Knowledge Base (Animal Classification):

1. All dogs are mammals: $\forall x\big(\text{Dog}(x) \Longrightarrow \text{Mammal}(x)\big)$
2. All mammals are animals: $\forall x\big(\text{Mammal}(x) \Longrightarrow \text{Animal}(x)\big)$
3. All animals breathe: $\forall x\big(\text{Animal}(x) \Longrightarrow \text{Breathes}(x)\big)$
4. Fido is a dog: Dog(Fido)
5. Fido barks (extra fact): Barks(Fido)

Goal: Prove that Fido breathes: Breathes(Fido)

(a) Conversion to CNF

1. $\neg\text{Dog}(x_1) \vee \text{Mammal}(x_1)$
2. $\neg\text{Mammal}(x_2) \vee \text{Animal}(x_2)$
3. $\neg\text{Animal}(x_3) \vee \text{Breathes}(x_3)$
4. Dog(Fido)
5. Barks(Fido)

Negated Goal: ¬Breathes(Fido)

(b) & (c) First-Order Resolution Step-by-Step Refutation Proof

Step	Clause 1 (From KB/Derived)	Clause 2 (From KB/Derived)	Unifier (MGU)	Resolvent
1	¬Breathes(Fido) *(Negated Goal)*	$\neg\text{Animal}(x_3) \vee \text{Breathes}(x_3)$	$\{x_3$ /Fido}	¬Animal(Fido)
2	¬Animal(Fido) *(from Step 1)*	$\neg\text{Mammal}(x_2) \vee \text{Animal}(x_2)$	$\{x_2$ /Fido}	¬Mammal(Fido)
3	¬Mammal(Fido) *(from Step 2)*	$\neg\text{Dog}(x_1) \vee \text{Mammal}(x_1)$	$\{x_1/\text{Fido}\}$	¬Dog(Fido)
4	¬Dog(Fido) *(from Step 3)*	Dog(Fido)	{ } *(Exact match)*	**Empty Clause []**

Q5. Forward and Backward Chaining
Knowledge Base (Horn Clauses):

- **Rules:**

R1: $A \wedge B \Longrightarrow C$
R2: $C \wedge D \Longrightarrow E$
R3: $F \Longrightarrow B$
R4: $G \Longrightarrow D$
R5: $E \wedge H \Longrightarrow I$

- **Facts:** A, F, G, H
- **Query:** I

Python Implementation:
Python

```
def forward_chaining(rules, facts, query):
    inferred = set(facts)
    agenda = list(facts)
    count = {rule: len(premise) for rule, premise in rules.items()}

    print("--- Forward Chaining Trace ---")
    while agenda:
        p = agenda.pop(0)
        print(f'Fact processed: {p}")
        if p == query: return True
        for rule, premise in rules.items():
            if p in premise:
                count[rule] -= 1
                if count[rule] == 0 and rule not in inferred:
                    print(f"  -> Rule triggered. Inferred: {rule}")
                    inferred.add(rule)
                    agenda.append(rule)
    return False

def backward_chaining(rules, facts, goal, depth=0):
    indent = "  " * depth
    print(f"{indent}Goal to prove: {goal}")
    if goal in facts:
        print(f"{indent}* {goal} is a known fact.")
        return True
    if goal not in rules:
        return False

    # Prove all premises for the goal
    print(f"{indent}* Using rule to prove {goal}: requires {rules[goal]}")
    for premise in rules[goal]:
        if not backward_chaining(rules, facts, premise, depth + 1):
            return False
    return True

rules = {'C': ['A', 'B'], 'E': ['C', 'D'], 'B': ['F'], 'D': ['G'], 'I': ['E', 'H']}
facts = ['A', 'F', 'G', 'H']
```

(a) Forward Chaining Trace

Data-driven reasoning (starts with facts, moves to conclusion):

Plaintext

```
Fact processed: A
Fact processed: F
  -> Rule triggered. Inferred: B
Fact processed: G
  -> Rule triggered. Inferred: D
Fact processed: H
Fact processed: B
  -> Rule triggered. Inferred: C
Fact processed: D
Fact processed: C
  -> Rule triggered. Inferred: E
Fact processed: E
  -> Rule triggered. Inferred: I
Fact processed: I (Query Found!)
```

(b) Backward Chaining Trace (AND-OR Goal Reduction)

Goal-driven reasoning (starts with goal, recursively checks dependencies):

Plaintext

```
Goal to prove: I
* Using rule to prove I: requires ['E', 'H']
  Goal to prove: E
  * Using rule to prove E: requires ['C', 'D']
    Goal to prove: C
    * Using rule to prove C: requires ['A', 'B']
      Goal to prove: A
      * A is a known fact.
      Goal to prove: B
      * Using rule to prove B: requires ['F']
        Goal to prove: F
        * F is a known fact.
    Goal to prove: D
    * Using rule to prove D: requires ['G']
      Goal to prove: G
      * G is a known fact.
  Goal to prove: H
  * H is a known fact.
Result: True
```

(c) Comparison

Feature	Forward Chaining	Backward Chaining
Direction	Data-driven (Bottom-up). Starts with facts.	Goal-driven (Top-down). Starts with the query.
Efficiency	Can be inefficient if it infers many facts irrelevant to the specific query.	Generally more efficient for specific queries, as it only explores relevant rules.

Feature	Forward Chaining	Backward Chaining
Suitability	Best when facts are coming in continuously and we need to monitor what conclusions can be drawn (e.g., event monitoring, configuration systems).	Best when there is a specific hypothesis or diagnosis to prove (e.g., medical diagnosis, debugging).